

Manual de Despliegue de CREApp

Guía de producción para servidores locales, virtualizados y entornos on-premises

Código del Documento:	CREAPP-OPS-01
Versión:	2.0 (Edición On-Premises)
Autor:	Juan Martín Morales
Repositorio:	https://github.com/juanm4morales/cre-app
Clasificación:	Documentación Técnica de Operaciones
Destinatarios:	Administradores de Sistemas, Infraestructura y DevOps
Fecha de Revisión:	30 de septiembre de 2026

Universidad Nacional de Cuyo — Sistema de Gestión Curricular y Equivalencias (CREApp)

Este documento describe la instalación, aseguramiento, operación y recuperación del servicio en infraestructura local.

Cobertura documental: servidores propios y Azure. Este documento no verifica infraestructura ni despliegues en ejecución.

Contenido

Propósito y alcance	1
1. Requisitos	1
2. Despliegue self-hosted	1
3. RespalDOS y restauración	5
4. Despliegue en Azure	6
5. Problemas frecuentes	7
6. Qué se comprobó	7
7. Referencias	7

Propósito y alcance

Autor: Juan Martín Morales **Repositorio:** github.com/juanm4morales/cre-app **Versión:** 3.0

CREApp es una SPA de React que consume una API Django sobre PostgreSQL. Hay **dos destinos válidos**: un servidor propio (self-hosted) o Microsoft Azure App Service. Ambos se describen aquí con el mismo nivel de detalle; elija según las reglas de red y operación de su institución.

Para el detalle técnico, ver [APENDICES.md](#): variables de entorno y [configuración efectiva](#), [restricciones de la receta](#), [seguridad y HSTS](#), [importadores y scripts](#) y [split-origin](#). Para la evidencia de las pruebas ejecutadas, ver [validation/ACCEPTANCE_REPORT.md](#).

1. Requisitos

Componente	Versión
Python	3.13 (venv)
Node.js	22 LTS (solo para compilar el frontend)
PostgreSQL	17 o superior soportado por el proveedor
Nginx	Cualquier versión estable reciente

Los valores de CPU, RAM y disco son una orientación inicial, no un mínimo medido. Dimensione según carga y disponibilidad reales.

2. Despliegue self-hosted

Topología: Nginx termina TLS y sirve la SPA; Django corre con Gunicorn en 127.0.0.1:8000; PostgreSQL acepta conexiones solo por loopback.

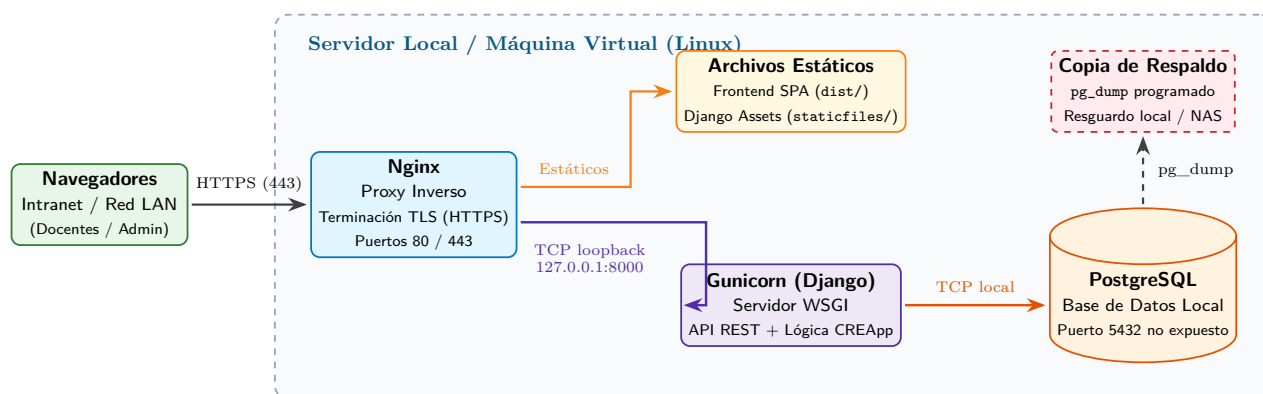


Figura 1. Arquitectura del servidor propio

Paso 1 — Usuarios, checkout y permisos

Se usan tres identidades: `creapp-build` compila, `creapp` ejecuta la aplicación y `www-data` sirve archivos. El código publicado nunca es escribible por el proceso web.

```

set -eu
SHA_APROBADO='<SHA_COMPLETO_APROBADO>' # commit revisado y aprobado
REF_APROBADA='<REF_APROBADA>'          # rama o tag que lo contiene

if ! getent group creapp >/dev/null; then sudo groupadd --system creapp; fi

```

```

id -u creapp >/dev/null 2>&1 || sudo useradd --system --gid creapp --groups www-data \
--home-dir /nonexistent --no-create-home --shell /usr/sbin/nologin creapp

if ! getent group creapp-build >/dev/null; then sudo groupadd --system creapp-build; fi
id -u creapp-build >/dev/null 2>&1 || sudo useradd --system --gid creapp-build \
--groups www-data --home-dir /var/lib/creapp-build --create-home \
--shell /usr/sbin/nologin creapp-build

sudo install -d -o root -g www-data -m 0750 /srv/creapp /srv/creapp/releases
sudo install -d -o root -g creapp -m 0750 /etc/creapp

RELEASE="/srv/creapp/releases/$SHA_APROBADO"
test ! -e "$RELEASE" || { echo 'El release ya existe' >&2; exit 1; }
sudo install -d -o creapp-build -g www-data -m 0750 "$RELEASE"

sudo -H -u creapp-build git clone --no-checkout \
https://github.com/juanm4morales/cre-app.git "$RELEASE"
sudo -H -u creapp-build git -C "$RELEASE" fetch --no-tags origin "$REF_APROBADA"
sudo -H -u creapp-build git -C "$RELEASE" checkout --detach "$SHA_APROBADO"
test "$(sudo -H -u creapp-build git -C "$RELEASE" rev-parse HEAD)" = "$SHA_APROBADO"

```

Paso 2 — PostgreSQL privado

```

sudo -u postgres psql

CREATE ROLE creapp_usr LOGIN;
\password creapp_usr
CREATE DATABASE creapp_db OWNER creapp_usr ENCODING 'UTF8';
\q

```

Fije listen_addresses = '127.0.0.1' en postgresql.conf y agregue al inicio de pg_hba.conf:

```

host    creapp_db    creapp_usr    127.0.0.1/32    scram-sha-256

sudo systemctl restart postgresql
sudo ss -ltnp | grep 5432    # debe mostrar solo 127.0.0.1

```

Nunca abra 5432 a la red. Para una base remota consulte el apéndice sobre TLS.

Paso 3 — Archivo de entorno

```

sudo install -o root -g creapp -m 0640 /dev/null /etc/creapp/creapp.env
sudoedit /etc/creapp/creapp.env

DEBUG=False
SECRET_KEY=<generado con python3 -c 'import secrets; print(secrets.token_urlsafe(50))'>
ALLOWED_HOSTS=<FQDN_REAL>
POSTGRES_DB=creapp_db
POSTGRES_USER=creapp_usr
POSTGRES_PASSWORD=<secreto>
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
CSRF_TRUSTED_ORIGINS=https://<FQDN_REAL>
CSRF_COOKIE_SECURE=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SAMESITE=Lax
SESSION_COOKIE_SAMESITE=Lax
SECURE_SSL_REDIRECT=True

```

ALLOWED_HOSTS lleva hostnames sin esquema. El archivo es de systemd, no un script de shell: use líneas `NOMBRE=valor` sin export ni comillas, y no haga `source`.

Paso 4 — Compilar, migrar y recolectar estáticos

El frontend se compila **antes** de `collectstatic`, porque el build entra en los estáticos de Django. Las migraciones son una operación separada y autorizada.

```
set -eu
RELEASE="/srv/creapp/releases/$SHA_APROBADO"

sudo -H -u creapp-build python3.13 -m venv "$RELEASE/.venv"
sudo -H -u creapp-build "$RELEASE/.venv/bin/pip" install --upgrade pip
sudo -H -u creapp-build "$RELEASE/.venv/bin/pip" install -r "$RELEASE/requirements.txt"

sudo -H -u creapp-build sh -c 'cd "$1/frontend" && npm ci && \
  VITE_API_URL=/api VITE_STATIC_BASE=/api/static/ npm run build' sh "$RELEASE"

# El código publicado no se modifica desde aquí en adelante.
sudo chown -R root:www-data "$RELEASE"
sudo find "$RELEASE" -type d -exec chmod 0750 {} +
sudo find "$RELEASE" -type f -exec chmod u=rwX,g=rX,o= {} +
sudo install -d -o creapp -g www-data -m 2750 "$RELEASE/backend/staticfiles"
```

Los comandos Django se ejecutan con el entorno de producción:

```
as_app() {
  sudo systemd-run --wait --pipe --collect \
    -p User=creapp -p Group=creapp -p SupplementaryGroups=www-data \
    -p WorkingDirectory="$RELEASE/backend" \
    -p EnvironmentFile=/etc/creapp/creapp.env \
    "$RELEASE/.venv/bin/python" "manage.py" "$@"
}

as_app check --deploy                # sin silenciar warnings
as_app migrate --plan                 # solo muestra el plan
as_app migrate --noinput              # requiere backup y ventana
as_app collectstatic --noinput
```

Paso 5 — Servicio systemd

/etc/systemd/system/creapp.service:

```
[Unit]
Description=CREApp Django Service
After=network.target postgresql.service

[Service]
Type=simple
User=creapp
Group=creapp
SupplementaryGroups=www-data
WorkingDirectory=/srv/creapp/current/backend
EnvironmentFile=/etc/creapp/creapp.env
UMask=0027
ExecStart=/srv/creapp/current/.venv/bin/gunicorn \
```

```
--workers 3 --bind 127.0.0.1:8000 config.wsgi:application
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
```

No inicie el servicio todavía: `current` se publica en el paso 7.

Paso 6 — Nginx

/etc/nginx/sites-available/creapp:

```
server {
    listen 80;
    server_name <FQDN_REAL>;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name <FQDN_REAL>;
    ssl_certificate <RUTA_FULLCHAIN_PEM>;
    ssl_certificate_key <RUTA_PRIVKEY_PEM>;
    client_max_body_size 25M;

    # Evita que el fallback SPA devuelva HTML en /api.
    location = /api { return 308 /api/; }

    location ^~ /api/static/ {
        alias /srv/creapp/current/backend/staticfiles/;
        expires 30d;
        access_log off;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location = /healthz {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        root /srv/creapp/current/frontend/dist;
        index index.html;
        try_files $uri $uri/ /index.html;
    }
}
```

Nginx debe sobrescribir `X-Forwarded-Proto`: Django confía en esa cabecera. Gunicorn y PostgreSQL no se exponen en el firewall.

Paso 7 — Publicar y verificar

Solo después de compilar, migrar y recolectar estáticos se publica el release:

```
set -eu
RELEASE="/srv/creapp/releases/$SHA_APROBADO"
test -d "$RELEASE/backend/staticfiles"
test ! -e /srv/creapp/current.next
sudo ln -s "$RELEASE" /srv/creapp/current.next
sudo mv -Tf /srv/creapp/current.next /srv/creapp/current
sudo nginx -t && sudo systemctl reload nginx
sudo systemctl enable --now creapp      # primera instalación
# sudo systemctl restart creapp         # en actualizaciones
```

Pruebas de humo con el dominio real:

```
curl -fsS -D - 'https://<FQDN_REAL>/healthz'          # 200 text/plain, cuerpo ok
curl -fsS -D - 'https://<FQDN_REAL>/api/auth/csrf'    # JSON, no HTML
curl -fsS -D - 'https://<FQDN_REAL>/api/'            # 308 hacia /api/
curl -fsS -D - 'https://<FQDN_REAL>/ruta-spa-profunda' # HTML de la SPA
```

Pruebe además el login y `/api/sadmin-creapp-panel/`. `/healthz` no consulta PostgreSQL.

Para revertir, apunte el symlink al release anterior y reinicie, solo si la base sigue siendo compatible. El código no se revierte junto con las migraciones.

3. Respaldos y restauración

Cree una cuenta y un `.pgpass` protegidos para que la contraseña no aparezca en comandos ni logs.

```
if ! getent group creapp-backup >/dev/null; then sudo groupadd --system creapp-backup; fi
id -u creapp-backup >/dev/null 2>&1 || sudo useradd --system --gid creapp-backup \
  --create-home --home-dir /var/lib/creapp-backup --shell /usr/sbin/nologin creapp-backup
sudo install -d -o creapp-backup -g creapp-backup -m 0700 /var/backups/creapp
sudo install -o creapp-backup -g creapp-backup -m 0600 /dev/null
↪ /var/lib/creapp-backup/.pgpass
sudoedit /var/lib/creapp-backup/.pgpass      # chmod 0600
```

```
127.0.0.1:5432:creapp_db:creapp_usr:<PASSWORD_REAL>
127.0.0.1:5432:creapp_restore_test:creapp_usr:<PASSWORD_REAL>
```

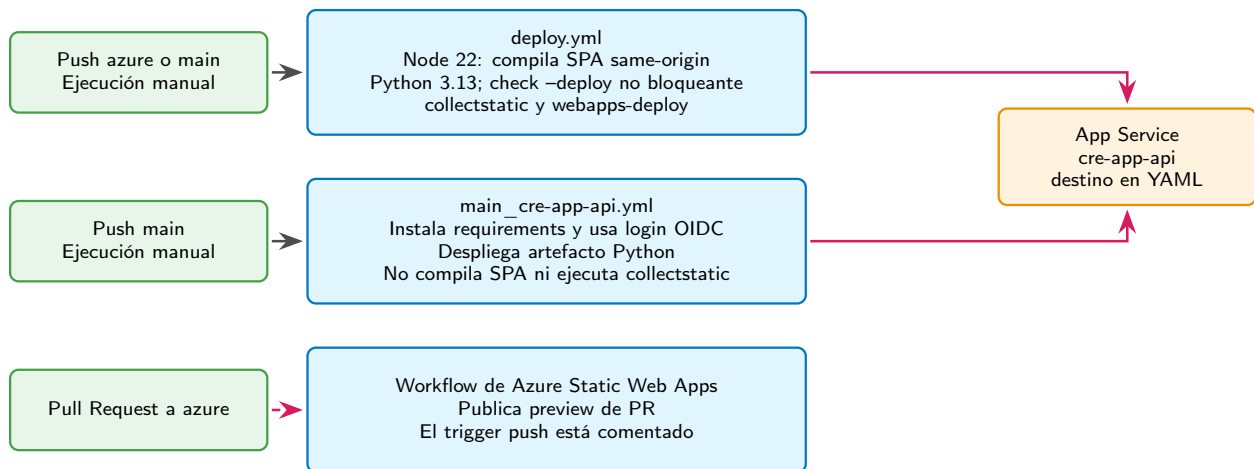
```
sudo -u creapp-backup env PGPASSFILE=/var/lib/creapp-backup/.pgpass \
  /bin/sh -c 'umask 077; exec /usr/bin/pg_dump -h 127.0.0.1 -U creapp_usr -F c \
  -f "/var/backups/creapp/creapp_$(date +%F_%H%M%S).dump" creapp_db'
```

Para restaurar, primero haga un ensayo en una base vacía y aislada. Nunca restaure sobre la base viva ni use `pg_restore --clean` como rollback.

```
sudo -u postgres createdb -O creapp_usr creapp_restore_test
sudo -u creapp-backup env PGPASSFILE=/var/lib/creapp-backup/.pgpass \
  pg_restore --exit-on-error --single-transaction -h 127.0.0.1 \
  -U creapp_usr -d creapp_restore_test /var/backups/creapp/creapp_<FECHA>.dump
```

La base de ensayo necesita su propia regla en `pg_hba.conf`, que se retira al terminar. Compare recuentos y datos representativos con el origen.

4. Despliegue en Azure



Un push a main puede iniciar ambos despliegues hacia el mismo App Service.
 Ninguno aplica migraciones; deploy.yml tampoco ejecuta tests y permite continuar si falla check -deploy.
 El commit activo, secretos, OIDC y estado del recurso se verifican en GitHub Actions y Azure Portal.

Figura 2. Flujos de despliegue en Azure

Paso 1 — PostgreSQL Flexible Server

1. Cree el servidor con autenticación por contraseña.
2. En el firewall autorice **todas** las direcciones de Outbound IP addresses y Additional outbound IP addresses del App Service. Autorizar solo una produce fallos intermitentes.
3. No active "permitir acceso público desde cualquier servicio de Azure": abre el acceso a cualquier tenant.
4. Con Private Endpoint, el App Service necesita VNet Integration en su propia subred y DNS privado que resuelva el nombre del servidor. Son dos recursos de red distintos.

Paso 2 — App Service

App Service (Linux) con runtime Python 3.13 y un plan acorde a la carga.

Paso 3 — Application Settings

```

DEBUG=False
SECRET_KEY=<secreto>
ALLOWED_HOSTS=<FQDN_APP_SERVICE>
POSTGRES_DB=creapp_db
POSTGRES_USER=creapp_usr
POSTGRES_PASSWORD=<secreto>
POSTGRES_HOST=<FQDN_POSTGRESQL>
POSTGRES_PORT=5432
CSRF_TRUSTED_ORIGINS=https://<FQDN_APP_SERVICE>
CSRF_COOKIE_SECURE=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SAMESITE=Lax
SESSION_COOKIE_SAMESITE=Lax
SECURE_SSL_REDIRECT=True
  
```

Los cambios de variables no recargan el proceso: pulse **Restart** en el portal. Las variables `VITE_*` son de compilación, no de runtime; `deploy.yml` compila con `/api` y `/api/static/`.

Paso 4 — Startup Command

```
gunicorn --bind=0.0.0.0:8000 --timeout 600 --chdir backend config.wsgi:application
```

Paso 5 — GitHub Actions

1. `deploy.yml` requiere `AZURE_WEBAPP_PUBLISH_PROFILE` y se activa con push a sus ramas de filtro o manualmente. Compila el frontend, ejecuta `collectstatic` y publica.
2. `main_cre-app-api.yml` despliega a la misma aplicación con otro paquete. Si ambas ramas de filtro se activan en un mismo push, se despliega dos veces. Use una sola rama de release.
3. El workflow de Static Web Apps tiene el push desactivado: solo previsualiza pull requests.

Los workflows no ejecutan pruebas ni migraciones, y `check --deploy` es no bloqueante. Un workflow en verde no acredita salud, migraciones ni base de datos.

Paso 6 — Verificación

El arranque en frío puede tardar 30 a 90 segundos: configure los tiempos de espera de monitoreo en 90 s o más. Valide `/healthz` y una ruta de API JSON.

5. Problemas frecuentes

Síntoma	Causa probable	Qué revisar
502 Bad Gateway	Gunicorn detenido	<code>systemctl status creapp</code> , <code>journalctl -u creapp -n 30</code> , <code>'ss -ltnp \</code>
400 Bad Request	Host no permitido	<code>ALLOWED_HOSTS</code> en <code>/etc/creapp/creapp.env</code> , luego reiniciar
403 CSRF	Origen no confiable	<code>CSRF_TRUSTED_ORIGINS</code> con el dominio exacto y acceso por HTTPS
Error de conexión a PostgreSQL	Credenciales o servicio	<code>systemctl status postgresql</code> , <code>psql -h 127.0.0.1 -U creapp_usr -d creapp_db</code>
404 de CSS o JS	Faltó el build o <code>collectstatic</code>	<code>readlink -f /srv/creapp/current</code> , <code>manifest</code> , alias de <code>/api/static/</code>
HTML en una ruta <code>/api</code>	Fallback SPA	Regla <code>location = /api</code> presente y <code>proxy_pass</code> correcto

6. Qué se comprobó

El harness de aceptación ejecutó el recorrido completo en un entorno aislado: Node 22, Python 3.13, PostgreSQL 17, Gunicorn y Nginx con TLS de prueba. Pasaron 54 pruebas frontend, 78 pruebas Django, las migraciones desde base vacía, la sesión con CSRF, el panel de administración, los estáticos y un ciclo de respaldo y restauración.

Ese recorrido **no** cubre Azure, una VM con systemd, DNS o certificados públicos, PostgreSQL remoto ni la política real de cookies del navegador. Repita estas verificaciones contra el destino que vaya a usar. Los detalles están en [validation/ACCEPTANCE_REPORT.md](#).

7. Referencias

- Django: [lista de comprobación para producción](#)
- Django: [archivos estáticos](#)

- Django: referencia de settings
- Gunicorn: despliegue con systemd
- Nginx: módulo de proxy inverso
- Nginx: servidores HTTPS
- PostgreSQL: pg_dump y pg_restore
- PostgreSQL: pg_hba.conf
- Azure: Python en App Service
- Azure: PostgreSQL Flexible Server
- Azure: VNet Integration
- systemd.service